Fashion Magazine Generator — Flask App

What Is Flask?

Flask is a **lightweight Python web framework**.

Let's break that down:

- **Python** → The programming language this project is written in.
- **Web framework** → A tool that helps you build websites or web applications.
- **Lightweight** → Simple, flexible, and easy to understand.

In this project, Flask:

- Runs a **local web server**
- Displays the Fashion Magazine interface in your browser

🎯 Task 1 — Implement `generate_image()` Using DALL·E

When you compared your notebook results with the web application, you probably noticed a big difference in image quality.

That's because:

- Your training dataset contains **very simple and limited images**
- The application uses **DALL·E**, a large-scale generative model trained on massive amounts of image–text data

---

What Is DALL·E?

DALL·E is a text-to-image model developed by OpenAI.

It:

- Takes a **text prompt** as input
- Generates a high-quality image matching that description
- Has been trained on a very large and diverse dataset
- Produces realistic, detailed, editorial-style images

In this application, DALL·E is used to generate **fashion editorial images** based on:

- The article title
- The image style description

Your task is to understand how the API call works and implement it step by step.

Your Goal

You will implement the `generate_image()` function. inside the app.py file

This function must:

1. Build a strong image prompt
2. Call the DALL·E API
3. Save the image to disk
4. Return the generated filename

Once you have done your changes continue with the setup to test your results

Setup

1. Create your `.env` file

In the `fashion_app/` folder create a file called `.env`:

```
OPENAI_API_KEY=your_litellm_key_here
```

2. Create virtual environment & install dependencies

We have provided you with two scripts one ending in ".sh" for macOS and one ending in ".ps1" for Windows. These scripts ensure you have python installed and create a virtual enviorment with all the necesarry libraries installed.

macOS: Make the script executable.

```
chmod +x setup.sh
```

Run the setup

```
./setup.sh
```

Windows: This script is made to be run in a powershell terminal. Make the script executable.

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
```

Run the setup

```
.\setup.ps1
```

3. Activate virtual environment

macOS:

```
source venv/bin/activate
```

Windows:

```
.\venv\Scripts\activate
```

You can tell that your terminal is in the virtual environment by seeing a "(venv)" in front of the current file in the terminal. It is essential to have your virtual environment active otherwise the app will not be able to find the necessary libraries.

4. Run the app

```
python3 app.py     # macOS
python app.py      # Windows
```

Open http://127.0.0.1:5001

How It Works

1. Fill in **Article Title**, **Image Styles**, and **Article Description**
2. Press ▶️ **Generate** — live progress streams in the sidebar
3. Article renders in the preview panel on the right
4. Press **Generate** again to refine — the last 5 sessions feed into the next generation for continuity

5. Press 🗑️ **Delete History** to start completely fresh

Project Structure

```
fashion_app/
├── app.py                  ← Flask app + full AI pipeline
├── templates/
│   ├── fashion_magazine.html ← Main UI
│   ├── article.html        ← Article wrapper
│   └── error.html          ← Error page
├── static/                 ← Static assets
├── Data/                   ← Place your .xls dataset here
├── output/                 ← Generated articles + images (auto-created)
├── history/                ← Session logs (auto-created)
├── requirements.txt
├── .env                    ← Your API key (never commit this!)
└── .gitignore
```

Key Design Decisions

- **Separate history logs** — `articles.json`, `images.json`, `text.json` tracked independently
- **Last 5 sessions** injected into each prompt for continuity (kept at 5 to avoid token limits)
- **3 retries + emergency fallback** — if HTML generation fails, a basic valid page is assembled from raw parts
- **SSE streaming** — live progress updates streamed to browser during generation
- **LangChain removed** — replaced with plain Python functions since the pipeline is fixed and predetermined